

PRACTICAL 8

Memory allocation

```
Open  ▾  +  *memory_allocation.c
~./OSlab

3 int main()
4 {
5     int i;
6     int *a = (int *)malloc(5 * sizeof(int));
7     if (a == NULL)
8     {
9         printf("malloc failed\n");
10        return 1;
11    }
12    for (i = 0; i < 5; i++)
13    {
14        a[i] = i + 1;
15    }
16    printf("Memory allocated using malloc:\n");
17    for (i = 0; i < 5; i++)
18    {
19        printf("%d ", a[i]);
20    }
21    printf("\n");
22    int *b = (int *)calloc(5, sizeof(int));
23    if (b == NULL)
24    {
25        printf("calloc failed\n");
26        free(a);
27        return 1;
28    }
29    printf("Memory allocated using calloc:\n");
30    for (i = 0; i < 5; i++)
31    {
32        printf("%d ", b[i]);
33    }
34    printf("\n");
35    a = (int *)realloc(a, 10 * sizeof(int));
36    if (a == NULL)
37    {
38        printf("realloc failed\n");
39        free(b);
40        return 1;
41    }
42    for (i = 5; i < 10; i++)
43    {
44        a[i] = i + 1;
45    }
46    printf("After realloc:\n");
47    for (i = 0; i < 10; i++)
48    {
49        printf("%d ", a[i]);
50    }
51    printf("\n");
52    free(a);
53    free(b);
54    printf("Memory freed successfully.\n");
55    return 0;
56 }
```

```
#include <stdio.h> #include
<stdlib.h> int main() { int i;

// malloc()
int *a = (int *)malloc(5 * sizeof(int));

if (a == NULL)
{
    printf("malloc failed\
n"); return 1;
}

for (i = 0; i < 5; i++)
{
    a[i] = i + 1;
}

printf("Memory allocated using malloc:\
n"); for (i = 0; i < 5; i++)
{
    printf("%d ", a[i]);
}
printf("\n");

// calloc()
int *b = (int *)calloc(5, sizeof(int));

if (b == NULL)
{
    printf("calloc failed\
n"); free(a);
    return 1;
}

printf("Memory allocated using calloc:\
n"); for (i = 0; i < 5; i++)
{
    printf("%d ", b[i]);
}
```

```
printf("\n");

// realloc()
a = (int *)realloc(a, 10 * sizeof(int));

if (a == NULL)
{
    printf("realloc failed\n");
    free(b);
    return 1;
}

for (i = 5; i < 10; i++)
{
    a[i] = i + 1;
}

printf("After realloc:\n");
for (i = 0; i < 10; i++)
{
    printf("%d ", a[i]);
}
printf("\n");
free(a);
free(b);
printf("Memory freed successfully.\n");
return 0;
}
```

```
sampath@MS-Desktop:~$ cd OSlab
sampath@MS-Desktop:~/OSlab$ gedit memory_allocation.c

** (gedit:511): WARNING **: 06:39:56.693: Could not load Peas repository: Typelib
0' not found

** (gedit:511): WARNING **: 06:39:56.693: Could not load PeasGtk repository: Type
on '1.0' not found
sampath@MS-Desktop:~/OSlab$ gcc -g memory_allocation.c -o memory_allocation
sampath@MS-Desktop:~/OSlab$ ./memory_allocation
Memory allocated using malloc:
1 2 3 4 5
Memory allocated using calloc:
0 0 0 0 0
After realloc:
1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
```

```
sampath@MS-Desktop:~/OSlab$ valgrind --version
valgrind-3.26.0
sampath@MS-Desktop:~/OSlab$ valgrind --leak-check=full ./memory_allocation
==7955== Memcheck, a memory error detector
==7955== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==7955== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==7955== Command: ./memory_allocation
==7955==
Memory allocated using malloc:
1 2 3 4 5
Memory allocated using calloc:
0 0 0 0 0
After realloc:
1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
==7955==
==7955== HEAP SUMMARY:
==7955==    in use at exit: 0 bytes in 0 blocks
==7955== total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==7955==
==7955== All heap blocks were freed -- no leaks are possible
==7955==
==7955== For lists of detected and suppressed errors, rerun with: -s
==7955== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Copy on write

```
Open  + *copy_on_write.c
~/OSlab

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #define SIZE 1000000
6 int main()
7 {
8     int *data;
9     pid_t pid;
10    data = (int *)malloc(SIZE * sizeof(int));
11    if (data == NULL)
12    {
13        printf("Memory allocation failed\n");
14        return 1;
15    }
16    for (int i = 0; i < SIZE; i++)
17    {
18        data[i] = 10;
19    }
20    printf("Parent PID: %d\n", getpid());
21    printf("Before fork: data[0] = %d\n", data[0]);
22    pid = fork();
23    if (pid < 0)
24    {
25        printf("Fork failed\n");
26        free(data);
27        return 1;
28    }
29    if (pid == 0)
30    {
31        printf("\nChild PID: %d\n", getpid());
32        printf("Before modification: data[0] = %d\n", data[0]);
33        data[0] = 999;
34        printf("After modification: data[0] = %d\n", data[0]);
35        printf("Child memory address: %p\n", (void *)&data[0]);
36        sleep(5);
37        free(data);
38        exit(0);
39    }
40    else
41    {
42        // Parent process
43        printf("\nParent after fork\n");
44        printf("Parent data[0] = %d\n", data[0]);
45        printf("Parent memory address: %p\n", (void *)&data[0]);
46        sleep(2);
47        printf("Parent data[0] after child modification = %d\n", data[0]);
48        wait(NULL);
49        free(data);
50    }
51    return 0;
52 }
```

```
sampath@MS-Desktop:~/OSlab$ gedit copy_on_write.c

** (gedit:7965): WARNING **: 06:53:49.931: Could not load Peas repository '1.0' not found

** (gedit:7965): WARNING **: 06:53:49.931: Could not load PeasGtk repository '1.0' not found
sampath@MS-Desktop:~/OSlab$ gcc -g copy_on_write.c -o copy_on_write
sampath@MS-Desktop:~/OSlab$ ./copy_on_write
Parent PID: 8694
Before fork: data[0] = 10

Parent after fork
Parent data[0] = 10
Parent memory address: 0x79c2fd02f010

Child PID: 8695
Before modification: data[0] = 10
After modification: data[0] = 999
Child memory address: 0x79c2fd02f010
Parent data[0] after child modification = 10
sampath@MS-Desktop:~/OSlab$ |
```

```
#include <stdio.h> #include <stdlib.h> #include <unistd.h> #include
```

```
<sys/wait.h> #define SIZE 1000000
```

```
int main() { int *data; pid_t pid;
```

```
// Allocate memory
```

```
data = (int *)malloc(SIZE * sizeof(int));
```

```
if (data == NULL)
```

```
{
```

```
    printf("Memory allocation failed\n");
```

```
    return 1;
```

```
}
```

```
// Initialize data
```

```
for (int i = 0; i < SIZE; i++)
```

```
{
```

```
    data[i] = 10;
```

```
}
```

```
printf("Parent PID: %d\n", getpid());
```

```
printf("Before fork: data[0] = %d\n",
data[0]);
```

```
pid = fork();

if (pid < 0)
{
    printf("Fork failed\n");
    free(data);
    return 1;
}

if (pid == 0)
{
    // Child process
    printf("\nChild PID: %d\n", getpid());

    printf("Before modification: data[0] = %d\n", data[0]);

    // Modify data
    data[0] = 999;

    printf("After modification: data[0] = %d\n", data[0]);

    printf("Child memory address: %p\n", (void *)&data[0]);

    sleep(5);

    free(data);
    exit(0);
}
else
{
    // Parent process
    printf("\nParent after fork\n");
    printf("Parent data[0] = %d\n", data[0]);
    printf("Parent memory address: %p\n", (void *)&data[0]);

    sleep(2);

    printf("Parent data[0] after child modification = %d\n",
        data[0]);
}
```

```
        wait(NULL);  
        free(data);  
    }  
  
    return 0;  
  
}
```